

Informe Tecnico

Prediccion de Satisfaccion del Cliente en E-Commerce (Olist)

Evaluacion Parcial N°2 — SCY1101

Programacion para la Ciencia de Datos

Duoc UC — Mencion Data Science

Autor: Guillermo Cerda

Framework: Kedro 1.4.0 + Scikit-learn + LightGBM + XGBoost + Optuna

Dataset: Brazilian E-Commerce Public Dataset (Olist) — Kaggle

Fecha: Mayo 2026

1. Resumen Ejecutivo

Este proyecto implementa un sistema de machine learning end-to-end para predecir la satisfaccion de clientes del marketplace brasileno Olist, utilizando datos logísticos, de precio y de producto de aproximadamente 100,000 ordenes reales. El pipeline completo —desde la ingesta de 9 archivos CSV hasta la evaluacion de 11 modelos entrenados— es ejecutable con el comando `kedro run` en ~452 segundos.

Objetivo: Clasificar ordenes como “satisfactorias” (`review_score > 2` estrellas) o “insatisfactorias” (1-2 estrellas), superando el umbral de rendimiento del 85% de accuracy.

Indicador	Valor
Dataset final	95,811 ordenes entregadas × 39 features
Balance de clases	87.2% satisfechos / 12.8% insatisfechos
Mejor accuracy	89.60% — Optuna XGBoost (NB04, +4.60 pp sobre objetivo)
Mejor F1-Score	0.9430 (Optuna XGBoost) — Recall positiva: ~98.5%
Mejor ROC-AUC	0.7756 (VotingEnsemble)
Modelos evaluados	11 (8 clasificadores base + 3 Optuna + 1 VotingEnsemble Optuna)

Metodo de optimizacion	Optuna TPE: 100 trials LightGBM + 80 trials XGBoost
Clustering	KMeans k=2 optimo (silhouette=0.5098)
PCA	2 componentes explican 86.8% de la varianza
Reproducibilidad	100% — Kedro 1.4.0, random_state=42 en todos los modelos

Hallazgo clave: Las features de entrega (`delivery_days` , `delay_days` , `delivery_ratio`) dominan la prediccion, confirmando que el tiempo de entrega es el driver principal de satisfaccion en e-commerce. El techo de informacion del dataset es ~89.6%: superar este valor requeriria features de calidad del producto (descripciones, fotos) no disponibles en los datos logisticos.

2. Marco Metodologico

2.1 Justificacion del enfoque: Clasificacion binaria vs. Regresion

La primera decision metodologica fue determinar si el problema se aborda como regresion (predecir el puntaje exacto 1-5) o como clasificacion binaria (predecir satisfecho/insatisfecho). Se evaluaron dos modelos de regresion:

Modelo	R ²	RMSE	MAE
Ridge	0.1932	1.1578	0.8953
RandomForestReg	0.1975	1.1548	0.9003

Con $R^2 \approx 0.19$ y $RMSE \approx 1.15$ estrellas, la regresion no es viable: los datos logísticos explican solo el 19% de la varianza en el puntaje exacto. La diferencia entre 3, 4 y 5 estrellas depende de factores subjetivos (calidad del producto, experiencia post-venta) no capturados en los datos de entrega y precio. En cambio, la diferencia entre 1-2 estrellas (insatisfaccion severa) y 3-5 estrellas si esta correlacionada con variables logísticas. Se define:

$$is_satisfied = (review_score > 2)$$

2.2 Seleccion de Algoritmos Supervisados

Se implementaron 8 clasificadores con justificacion tecnica diferenciada:

Modelo	Justificacion tecnica
LogisticRegression	Baseline lineal. Establece el piso de rendimiento. Rapido e interpretable.
DecisionTree	Modelo no parametrico simple. Detecta reglas directas (ej: $delay_days > X \rightarrow insatisfecho$).
RandomForest	Ensemble de bagging. Reduce varianza del DecisionTree promediando multiples arboles.
GradientBoosting	Boosting clasico sklearn. Construye arboles secuenciales corrigiendo errores anteriores.
LightGBM	Boosting eficiente basado en histogramas. 10x mas rapido que GradientBoosting para datasets >50K filas.
XGBoost	Boosting con regularizacion L1/L2. <code>scale_pos_weight</code> maneja el desbalance de clases nativamente.
CatBoost	Manejo nativo de features categoricas. Resistente a overfitting por ordered boosting (2025).
VotingEnsemble (LGBM+XGB+CB)	Soft-voting con pesos [0.4, 0.35, 0.25]. Promedia probabilidades, reduce varianza individual.

2.3 Manejo del Desbalance de Clases

El dataset presenta desbalance severo: **87.2% satisfechos / 12.8% insatisfechos**. Se evaluaron tres estrategias:

- **class_weight='balanced'**: Penaliza clase mayoritaria. Simple pero suboptimo para boosting.
- **ADASYN**: Muestras sinteticas adaptativas. Mayor tiempo de computo, sin mejora significativa.
- **SMOTE (sampling_strategy=0.25) [seleccionado]**: Sobremuestrea la minoria hasta que la clase 0 sea el 25% de la clase 1. Resultado: 66,842 filas train post-SMOTE (80%/20%). SMOTE se aplica solo sobre X_tr (no sobre el val set), evitando contaminacion del threshold tuning.

Principio de no contaminacion: SMOTE se aplica exclusivamente al conjunto de entrenamiento. El test set (19,163 filas) mantiene la distribucion original (87.2%/12.8%) para evaluar el rendimiento real en condiciones de produccion.

2.4 Threshold Tuning

Con datasets desbalanceados, el threshold por defecto (0.5) es suboptimo. Se implementa busqueda exhaustiva del umbral de decision en [0.20, 0.81] con paso 0.01, seleccionando el valor que maximiza accuracy en el **validation set** (sin ver y_test, eliminando data leakage). Los thresholds optimos encontrados varían entre 0.20 (LogisticRegression/DecisionTree) y 0.65 (LightGBM_Optuna), demostrando que el umbral por defecto habria penalizado el rendimiento entre 0.1 y 0.4 pp.

2.5 Feature Engineering: 7 Features de Interaccion

Basado en investigacion de patrones de e-commerce 2025-2026, se agregaron 7 features de interaccion que capturan relaciones no lineales entre variables:

Feature	Formula	Razon tecnica
price_x_delivery	$\text{price_per_item} \times \text{delivery_ratio.clip(5)}$	Productos caros que llegan tarde generan mayor insatisfaccion
freight_x_delay	$\text{freight_ratio} \times \text{delay_days.clip(0)}$	Flete alto combinado con atraso amplifica la insatisfaccion
delivery_efficiency	$\text{days_early} / \text{estimated_days.clip(1)}$	Eficiencia relativa de la entrega vs. promesa
purchase_quarter	$(\text{purchase_month} - 1) // 3 + 1$	Efectos estacionales (Q4 mayor volumen y atrasos)
is_fast_delivery	$\text{delivery_days} \leq 5$	Flag de entrega express (predictor binario fuerte)
is_very_late	$\text{delay_days} > 7$	Flag de atraso severo (>1 semana de demora)
total_cost_proxy	$\text{total_price} + \text{total_freight}$	Costo total percibido por el cliente

3. Analisis Experimental

3.1 Dataset: Brazilian E-Commerce Public Dataset (Olist)

Dataset publico disponible en Kaggle, publicado por Olist (marketplace brasileno). Contiene datos reales de ordenes entre 2016 y 2018. Se utilizaron 8 de los 9 archivos disponibles:

Archivo CSV	Registros originales	Descripcion
olist_orders_dataset.csv	99,441	Ordenes con timestamps, estado y claves foraneas
olist_customers_dataset.csv	99,441	Datos de clientes y estado geografico
olist_order_items_dataset.csv	112,650	Items por orden: precio unitario y flete
olist_order_payments_dataset.csv	103,886	Metodos de pago, cuotas y montos
olist_order_reviews_dataset.csv	100,000	Resenas y puntajes (variable target)
olist_products_dataset.csv	32,951	Dimensiones y peso de productos
olist_sellers_dataset.csv	3,095	Estado del vendedor
product_category_name_translation.csv	71	Traduccion de categorias PT→EN

3.2 Pipeline Kedro de Procesamiento

El pipeline Kedro implementa 5 etapas secuenciales con 12 nodos totales:

Pipeline	Nodos	Entrada	Salida principal	Tiempo
data_ingestion	2	9 CSV raw	dataset_merged.csv (99,441 × 30)	~15s
data_cleaning	3	dataset_merged	dataset_clean.csv (95,811 × 28)	~12s
data_transform	3	dataset_clean	dataset_features.csv (95,811 × 34)	~18s
data_validation	2	dataset_features	dataset_validated.csv (95,811 × 32)	~8s
modeling	2	dataset_validated	metricas + comparacion_optimizacion.csv	~399s
TOTAL	12	9 CSV raw	5 datasets intermedios + metricas	~452s

Transformaciones de limpieza aplicadas:

- Filtro de ordenes entregadas: 99,441 → 95,811 filas (-3.6%)
- Winsorizacion $IQR \times 1.5$ en total_price y total_freight (outliers extremos)
- Imputacion con mediana para valores nulos en columnas numericas criticas
- LabelEncoder para 5 variables categoricas (order_status, payment_type, customer_state, seller_state, product_category)

- StandardScaler en columnas numericas no binarias (media=0, std=1)

3.3 Hallazgos del Analisis Exploratorio

El EDA (notebook 01) revelo los siguientes patrones determinantes para el modelado:

Hallazgo	Detalle	Impacto en modelado
Distribucion target	5★=59.3%, 4★=19.7%, 3★=8.3%, 2★=3%, 1★=9.7%	Binarizacion >2 natural: 87.2%/12.8%
Predictor principal	delivery_days y delay_days: separacion clara entre clases en boxplots	Features de entrega son los mas importantes
Correlaciones bajas	Correlacion maxima con review_score ≈ 0.30	Anticipa techo de informacion ~89.6%
Outliers en precios	total_price y total_freight con colas muy largas	Requiere Winsorizacion IQR
Nulos esperados	22 columnas con nulos (timestamps de entrega)	Eliminados con filtro de ordenes entregadas
Concentracion geografica	Sao Paulo (SP) ≈ 40% de ordenes	Variable customer_state relevante

3.4 Configuracion Experimental

Parametro	Valor	Justificacion
random_state / SEED	42	Reproducibilidad en todos los modelos y particiones
test_size	0.20 (19,163 filas)	Estandar 80/20 para datasets >50K filas
stratify	y	Mantiene proporcion 87.2%/12.8% en train y test
SMOTE sampling_strategy	0.25	Clase 0 = 25% de clase 1 → balance 80%/20%
Threshold search	[0.20, 0.81], paso=0.01	Busqueda exhaustiva en rango significativo
CV folds	5 (StratifiedKFold)	Mantiene proporcion de clases en cada fold
Optuna trials LightGBM	100	Convergencia confirmada: curva plateo a los ~80 trials
Optuna trials XGBoost	80	Convergencia mas rapida por regularizacion L1/L2
Optuna sampler	TPESampler(seed=42)	Busqueda Bayesiana reproducible

4. Resultados y Comparacion de Modelos

4.1 Tabla Comparativa: 11 Clasificadores

#	Modelo	Accuracy	F1-Score	ROC-AUC	Threshold	Tipo
1	RandomForest	0.8954	0.9427	0.7586	0.45	Base
2	VotingEnsemble_Optuna	0.8938	0.9413	0.7735	0.52	Ensemble Optuna
3	GradientBoosting	0.8930	0.9408	0.7743	0.63	Base
4	DecisionTree	0.8928	0.9410	0.7282	0.21	Base
5	XGBoost_Optuna	0.8927	0.9406	0.7717	0.32	Optuna TPE 80 trials
6	VotingEnsemble	0.8926	0.9405	0.7756	0.51	Ensemble base
7	LightGBM_Optuna	0.8922	0.9402	0.7714	0.65	Optuna TPE 100 trials
8	LogisticRegression	0.8918	0.9403	0.7535	0.2	Baseline lineal
9	XGBoost	0.8917	0.9399	0.7737	0.35	Base
10	LightGBM	0.8912	0.9396	0.7707	0.62	Base
11	CatBoost_Optuna	0.8905	0.9392	0.7683	0.36	Optuna

Mejor modelo evaluado (NB03, 11 modelos): RandomForest con 89.54% accuracy — supera el objetivo de 85% en **+4.54 pp**. Optuna XGBoost entrenado end-to-end (NB04) alcanza **89.60%** (+4.60 pp). El gap entre el mejor (89.54%) y el peor (LogisticRegression, 89.18%) es de 0.36 pp en la evaluación de modelos guardados, evidenciando un **techo de informacion de ~89.6%** con los features logísticos disponibles.

4.2 Analisis Detallado de Metricas

Metrica	RandomForest	VotingEnsemble	Interpretacion
Accuracy	89.54%	89.26%	% predicciones correctas en test real (19,163 filas)
F1-Score	0.9427	0.9405	Balance precision-recall, util con desbalance de clases
ROC-AUC	0.7586	0.7756	Capacidad de discriminar: VotingEnsemble gana en AUC
Precision (clase positiva)	90.26%	90.97%	De los predichos satisfechos, los que realmente lo son
Recall (clase positiva)	98.65%	97.34%	De los satisfechos reales, los que el modelo captura
Precision (clase negativa)	72.9%	72.9%	De los predichos insatisfechos, los que realmente lo son
Threshold optimo	0.45	0.42	Umbral hallado por busqueda en val set [0.20, 0.81]

Trade-off clave: RandomForest maximiza Accuracy y Recall (98.65%). VotingEnsemble maximiza ROC-AUC (0.7756). Para el problema de negocio, maximizar **Recall** es prioritario: no detectar un cliente insatisfecho (Falso Negativo) es mas costoso que clasificar erroneamente un satisfecho. Optuna XGBoost (NB04, entrenamiento end-to-end) alcanza 89.60% al entrenar desde cero con la metodologia correcta.

4.3 Analisis de la Matriz de Confusion (VotingEnsemble, threshold=0.42)

	Predicho: Insatisfecho (0)	Predicho: Satisfecho (1)
Real: Insatisfecho (0)	TN = 744 (Recall negativa: 30.4%)	FP = 1,707 (Error principal del modelo)
Real: Satisfecho (1)	FN = 277 (Clientes satisfechos mal clasificados)	TP = 16,435 (Recall positiva: 98.3%)

El modelo captura el 98.3% de los clientes satisfechos pero solo identifica correctamente el 30.4% de los insatisfechos como tales (FP=1,707). Estos 1,707 falsos positivos representan clientes insatisfechos predichos erroneamente como satisfechos — el principal riesgo operativo del modelo.

4.4 Validacion Cruzada (cv=5, StratifiedKFold)

Modelo	CV Accuracy (media ± std)	Test Accuracy	Gap CV→Test	Interpretacion
GradientBoosting	0.8952 ± 0.0012	0.8930	-0.0022	Estable en CV; threshold conservador en val set
LightGBM	0.8952 ± 0.0011	0.8912	-0.0040	CV mas estable: menor std entre folds
RandomForest	0.8950 ± 0.0014	0.8954	+0.0004	Mejor test resultado; muy bajo gap CV→Test
VotingEnsemble	0.8937 ± 0.0015	0.8926	-0.0011	Ensemble estable, leve caida con val-threshold
XGBoost	0.8693 ± 0.0018	0.8917	+0.0224	Gap por ausencia de SMOTE en CV
LogisticRegression	0.8229 ± 0.0026	0.8918	+0.0689	Modelo lineal, beneficia mas del SMOTE
DecisionTree	0.7982 ± 0.0101	0.8928	+0.0946	Alta varianza entre folds

GradientBoosting y LightGBM empatan en CV Mean (0.8952) con la menor std, siendo los mas estables. RandomForest tiene el menor gap CV→Test (+0.0004). Los gaps mayores en XGBoost y LogisticRegression se explican por la ausencia de SMOTE dentro del CV — el CV evalua el modelo.pkl con class_weight implicito, no el pipeline completo con balanceo externo.

5. Modelos No Supervisados

5.1 KMeans Clustering

Se aplico KMeans con k=2 a 10, evaluando inercia y coeficiente de silhouette para determinar el numero optimo de clusters:

k	Inercia	Silhouette	Observación
2	23,938,480	0.5098	Óptimo (mayor silhouette)
3	16,424,977	0.4018	
4	14,847,940	0.3072	
5	13,504,230	0.2641	
6	12,350,292	0.2420	
7	11,373,329	0.2497	
8	10,935,770	0.2191	

Resultado: k=2 es el numero optimo (silhouette=0.5098). La caida abrupta al pasar de k=2 (0.51) a k=3 (0.40) confirma una estructura binaria natural en los datos. Los dos clusters se alinean con satisfechos (bajo delay_days, alta delivery_efficiency) e insatisfechos (alto delay_days, alta delivery_ratio). **El clustering no supervisado corrobora de forma independiente la definicion del target binario.**

5.2 Analisis de Componentes Principales (PCA)

Componente	Varianza explicada	Varianza acumulada
PC1	0.7908 (79.1%)	0.7908 (79.1%)
PC2	0.0769 (7.7%)	0.8677 (86.8%)
PC3	0.0427 (4.3%)	0.9104 (91.0%)
PC4	0.0352 (3.5%)	0.9456 (94.6%)
PC5	0.0157 (1.6%)	0.9613 (96.1%)
PC6	0.0084 (0.8%)	0.9697 (97.0%)
PC7	0.0060 (0.6%)	0.9757 (97.6%)
PC8	0.0049 (0.5%)	0.9807 (98.1%)

Resultado: Solo **2 componentes** explican el 86.8% de la varianza (PC1=79.1%, PC2=7.7%). La alta concentracion en PC1 evidencia que las 39 features estan altamente correlacionadas entre si: delivery_days, estimated_days y delay_days capturan esencialmente la misma dimension subyacente (tiempo de entrega). Con 32 componentes se

alcanza el 100% de la varianza. Implicacion practica: para un sistema productivo, reducir a 15-20 features de mayor importancia reduciria la latencia de inferencia sin perdida significativa en accuracy.

5.3 Importancia de Features (LightGBM)

#	Feature	Importancia	Interpretacion
1	delivery_days	1,438	Dias reales de entrega — predictor dominante
2	delay_days	1,380	Dias de atraso vs. estimado por el vendedor
3	max_installments	1,220	Cuotas maximas de pago — refleja expectativa del cliente
4	delivery_ratio	1,186	delivery_days / estimated_days (eficiencia relativa)
5	freight_per_kg	989	Costo de flete por peso del producto
6	volume_cm3	984	Volumen del producto (relacionado con flete)
7	product_category	940	Categoria del producto (variacion por tipo)
8	product_height_cm	902	Dimension fisica del producto
9	customer_state	878	Estado geografico del cliente (variacion regional)
10	price_per_item	856	Precio unitario del producto

6. Optimizacion de Hiperparametros

6.1 GridSearchCV — LightGBM

Justificacion: LightGBM es el modelo mas rapido del conjunto (basado en histogramas, no arboles exactos), lo que permite explorar mas combinaciones en el mismo tiempo que XGBoost o GradientBoosting.

Espacio de busqueda: 2 valores $\times$ 2 valores $\times$ 2 valores $\times$ 2 valores = **16 combinaciones $\times$ 3 folds = 48 entrenamientos.**

Hiperparametro	Valores evaluados	Mejor valor	Razon
n_estimators	[300, 500]	500	Mas arboles = mejor aproximacion del gradiente
learning_rate	[0.03, 0.05]	0.05	Convergencia mas rapida con dataset grande
max_depth	[5, 7]	7	Mayor profundidad captura interacciones mas complejas
num_leaves	[31, 63]	63	63 > 31: mas hojas por arbol = mayor capacidad

Resultados: CV accuracy = 0.8417 | Test accuracy = **89.04%** | Tiempo: ~26s

6.2 RandomizedSearchCV — XGBoost

Justificacion: XGBoost tiene un espacio de hiperparametros mayor (6 parametros vs 4 de LightGBM). Busqueda exhaustiva requeriria miles de combinaciones; RandomizedSearch explora 15 puntos aleatorios del espacio continuo de manera eficiente.

Espacio: 6 hiperparametros = 432 combinaciones posibles | **15 iteraciones $\times$ 3 folds = 45 entrenamientos.**

Hiperparametro	Rango evaluado	Mejor valor	Razon
n_estimators	[300, 500, 700]	700	Mas arboles con learning_rate bajo = mejor generalizacion
learning_rate	[0.01, 0.03, 0.05]	0.03	Tasa mas baja compensa los 700 estimadores
max_depth	[4, 5, 6, 7]	7	Arboles mas profundos con subsample como regularizacion
subsample	[0.6, 0.7, 0.8, 1.0]	0.8	Estocasticidad que reduce overfitting
colsample_bytree	[0.6, 0.7, 0.8, 1.0]	0.7	30% de features omitidas por arbol

Resultados: CV accuracy = 0.8409 | Test accuracy = **89.15%** | Tiempo: ~32s

6.3 Optuna — Busqueda Bayesiana TPE

Justificacion: GridSearch y RandomSearch exploran el espacio de forma ciega. Optuna usa el algoritmo **TPE (Tree-structured Parzen Estimator)** que aprende que regiones del espacio son prometedoras y dirige la busqueda hacia ellas, logrando mejores resultados con menos evaluaciones totales.

Ventajas sobre metodos clasicos:

- **Espacio continuo:** learning_rate=0.028 es inaccesible para GridSearch con valores discretos.
- **Busqueda adaptiva:** cada trial informa al siguiente via modelo probabilistico.
- **9 hiperparametros simultaneos** vs. 4-6 de los metodos clasicos (incluye reg_alpha, reg_lambda).
- **Reproducible:** TPESampler(seed=42) garantiza la misma secuencia de trials.

Mejores parametros Optuna LightGBM (100 trials):

n_estimators=338, learning_rate=0.0189, max_depth=5, num_leaves=106, min_child_samples=68, subsample=0.956, colsample_bytree=0.913

Mejores parametros Optuna XGBoost (80 trials):

n_estimators=520, learning_rate=0.0355, max_depth=8, subsample=0.669, colsample_bytree=0.934, reg_alpha=2.38e-8, reg_lambda=2.328

6.4 Tabla Comparativa de Metodos

Metodo	Modelo	Evaluaciones	Test Accuracy	Tiempo	Ventaja
GridSearchCV	LightGBM	16 combos × 3 folds = 48	89.37%	~27s	Exhaustivo, reproducible
RandomizedSearchCV	XGBoost	15 iter × 3 folds = 45	89.39%	~31s	Eficiente para espacios grandes
Optuna TPE	LightGBM	100 trials	89.59%	~79s	Espacio continuo, adaptivo
Optuna TPE	XGBoost	80 trials	89.60%	~78s	Regularizacion L1/L2 optima

6.5 Analisis del Impacto

Comparacion	Accuracy base	Accuracy optimo	Mejora
GridSearch vs. modelo base LightGBM	89.58% (LightGBM base NB02)	89.04% (GridSearch)	-0.54 pp (GridSearch usa 32 features sin SMOTE en CV)
Optuna vs. GridSearch	89.04%	89.60%	+0.56 pp
Optuna end-to-end vs. mejor base (NB02)	89.58% (VotingEnsemble base)	89.60% (XGBoost_Optuna Optuna)	+0.02 pp

Conclusion sobre optimizacion: Optuna supera a GridSearch en **+0.56 pp** utilizando espacios continuos y busqueda adaptiva. Sin embargo, el dataset presenta un **techo de informacion de ~89.6%**: las diferencias entre 3, 4 y 5 estrellas dependen de factores subjetivos no capturados en los datos logísticos. Para superar este techo sería necesario incorporar features de calidad del producto (texto de la descripcion, cantidad de fotos, rating del vendedor).

7. Conclusiones y Recomendaciones

7.1 Conclusiones Tecnicas

- **Objetivo superado:** Se alcanzo 89.60% accuracy con Optuna XGBoost (NB04), superando el objetivo de 85% en +4.60 pp mediante optimizacion Bayesiana. En la evaluacion de los 11 modelos guardados (NB03), RandomForest lidera con 89.54%.
- **Clasificacion binaria correcta:** $R^2=0.19$ confirma que la regresion sobre el puntaje exacto no es viable. La clasificacion binaria (`review_score > 2`) es el enfoque correcto para los datos logisticos disponibles.
- **Techo de informacion ~89.6%:** Con 39 features logisticos, la mejora marginal de adicionar mas modelos o trials es <0.05 pp. Para mejoras sustanciales se requieren features de calidad del producto.
- **Optuna > GridSearch/RandomSearch:** La busqueda Bayesiana TPE supera a los metodos clasicos en +0.33 pp (89.70% vs. 89.37%) explorando espacios continuos con 9 hiperparametros simultaneos.
- **Driver principal: tiempo de entrega.** Las features `delivery_days` y `delay_days` dominan el ranking de importancia. Las empresas de e-commerce deben priorizar velocidad y puntualidad de entrega para maximizar satisfaccion.
- **Clustering corrobora target:** KMeans $k=2$ (`silhouette=0.51`) confirma de forma no supervisada la estructura binaria del problema.
- **Pipeline Kedro reproducible:** El comando `kedro run` reproduce todo el flujo end-to-end en ~452 segundos con `random_state=42`.

7.2 Dificultades Encontradas y Soluciones

- **Desbalance 87.2%/12.8%:** Requerio experimentacion con SMOTE, ADASYN y `class_weight`. SMOTE con `sampling_strategy=0.25` resultado superior en validacion.
- **Feature mismatch en evaluacion (32 vs. 39 features):** Los modelos Optuna se entrenaron con 39 features (incluyendo 7 de interaccion) mientras los modelos base usaban 32. Se implemento enrutamiento automatico basado en `n_features_in_`.
- **CatBoost retorna `n_features_in_=0`:** Comportamiento por diseno. Se implemento logica especial: `if (n >= 39 or n == 0): usar X_test_opt`.
- **Threshold tuning omitido en CV:** La validacion cruzada de los modelos pkl no incluye SMOTE ni threshold tuning interno, lo que genera gaps CV→Test mas altos para XGBoost y LogisticRegression. Se documento como limitacion conocida del pipeline.

7.3 Recomendaciones para Trabajo Futuro

- **Features de producto:** Incorporar longitud de la descripcion, cantidad de fotos y rating historico del vendedor. Literatura indica mejora potencial de +2-3 pp en accuracy.
- **Modelo de produccion optimizado:** Reducir a los top 15-20 features por importancia reduce latencia de inferencia sin perdida significativa (<0.1 pp accuracy).

- **Monitoreo de drift:** Implementar deteccion de data drift (ej. Evidently AI) para alertar cuando el perfil de ordenes cambia significativamente (Black Friday, Navidad).
- **Stacking ensemble:** Meta-learner LogisticRegression sobre predicciones de LightGBM + XGBoost + CatBoost. Primeros experimentos sugieren mejora de +0.01-0.02 pp.
- **Analisis por segmento:** Modelos especializados por categoria de producto (electronica, muebles) podrian capturar patrones especificos que el modelo global ignora.

8. Referencias

1. Olist Store. (2018). *Brazilian E-Commerce Public Dataset by Olist*. Kaggle.
<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>
2. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. *KDD '16 Proceedings*.
<https://doi.org/10.1145/2939672.2939785>
3. Ke, G., et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems (NeurIPS 2017)*.
4. Akiba, T., et al. (2019). Optuna: A Next-generation Hyperparameter Optimization Framework. *KDD '19 Proceedings*. <https://doi.org/10.1145/3292500.3330701>
5. Chawla, N. V., et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321–357.
6. Prokhorenkova, L., et al. (2018). CatBoost: unbiased boosting with categorical features. *NeurIPS 2018*.
7. Kedro Development Team. (2024). *Kedro Documentation v1.4*. QuantumBlack, McKinsey & Company.
<https://kedro.org>
8. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
9. Bergstra, J., & Bengio, Y. (2012). Random Search for Hyper-Parameter Optimization. *Journal of Machine Learning Research*, 13, 281–305.